

A Jai-inspired systems language in Rust. Incremental salsa front end, lossless CST, typed SSA mid-end, two execution engines held byte-identical by a differential harness.

6/6 gates green 981 tests ADR-0097 latest W7 OPEN · String, Sort

TESTS	CORPUS	ADRS	DIAGNOSTICS	EDITOR CHECKS
984	193	105	109	166
workspace, all passing	jr files, both engines	0001 to 0105, immutable	codes, E0279 next free	Neovim, verified not gated

The vertical slice, and what it costs to be honest about it

EXIT CRITERIA DONE hello.jr runs in the VM DONE compiles to a native binary DONE rustc-grade diagnostics DONE formatter round-trips the corpus DONE editor integration (Neovim) OPEN verified Linux x86-64 CI run The last one needs a push. Configured, never run, so it is a decision rather than a technical gap.	HOW CORRECTNESS IS ESTABLISHED Both engines run every corpus program and must agree, on output and on trap wording, from one shared formatter. A plan's stated reason is checkable. W4.5 was scheduled after W4 because exhaustiveness "wants comptime type info". Three greps and a two-line program showed it does not: the enum member set is already in the pool during checking. The wave moved forward and the table records the amendment — a wave order resting on a dependency that does not exist is a plan contradicting itself. Check the query a claim is about. "The arithmetic around a #run is not folded" was true of the built MIR and false of the optimized one — and the corpus only ever snapshots the built query, so nothing the tests display would have shown it. A claim about optimisation needs a probe of the optimized body. A well-typed placeholder is invisible to every gate this project has. <code>t := Point; —</code> a type bound to a local — type-checked cleanly and both engines exited 0, storing an undefined value into a slot of a type with no runtime layout at all. Three of these now, and each was found by asking a question no test asks: what does this lower to? The corpus checks exit codes and snapshots the built MIR; a construct that is legal, silent and unread sits outside all of it. A test naming an unimplemented thing has a one-wave shelf life. The refused-body test has now had its construct replaced twice, both times because the wave after it implemented the gap it named. It uses something refused by design now. And one of this wave's own new tests passed vacuously until it was teeth-checked.
--	---

The language today

WORKS, END TO END Full integer tower, bool, string, pointers float32 and float64, plain IEEE-754, no traps struct, nominal, one level union, nominal, untagged — a cross-field read reinterprets variant — a tagged union: a wrong-case read traps, switch destructures enum and enum_flags, namespaced, bare dot-member, switch cases Fixed arrays and views, bounds-checked; a length may name a constant cast and xx from context; operator overloading Trapping arithmetic, wrapping variants, bitwise if, else, while, for, break, continue, defer, using switch with exhaustiveness checking over an enum; else Multiple returns, named args, literal defaults import, foreign, system_library, #scope_module, #expand macros that splice, #modify predicates, #bake_arguments, @note metadata \$T procedures, Box(\$T) structs, and \$N comptime-value parameters — complete, including [N]T sized by a \$N; all instantiated in both engines Compile-time run at file scope or in a body, across files; type_info(), Any, #insert, #code A type as a compile-time value: <code>T :: Point</code> aliases one, usable anywhere <code>Point</code> is A type in a runtime position is refused — it has no representation to store Traps name their source line and the live call chain Bounds checks, strippable by build setting or #no_abc context, a hidden parameter passed by pointer Procedures as values: call, pass, return, struct field null, a context-typed pointer literal; malloc and free An allocator in the context, installed and called through Pointer offset <code>p + n</code>, <code>n + p</code>, <code>p - n</code> — element-scaled, unchecked push_context, a block with its own copy of the context talloc / reset_temporary_storage — a per-context bump arena	ABSENT (WAVE) pointer difference <code>p - q</code>, deferred percent on floats, is_nan, math (W7) a recursive variant; eliding the check in an arm an explicit backing type a length needing evaluation; array literals unary, index and call overloading transmute; float printing patterns, ranges, guards; a jump table #must; a multi-result call in a return a reader for @note: the message loop (W6) #expand macros; inference through Box(\$T); a length needing arithmetic a cross-file #run value; a Code value (declined) a chain <code>B :: A</code>; comparing types; Type as an annotation a per-frame line; inlined frames (none exist) a per-index #no_abc; any other build setting a cross-file or foreign proc value; comparing one cast to a pointer the difference <code>p - q</code>; <code>p[n]</code>; ordering hands out *u8 only; alignment; a growable region
---	--

No error-handling model yet — ADR-0008 reserves the slot and the first half exists (several return values); #must is owed its own ADR. No GC and no RAI, which is a design value rather than a gap. A union is untagged, so reading a field other than the one last written reinterprets bits: documented in three places because it cannot be diagnosed. No indirect calls, which is the single largest gap in the project and is what blocks the rest of W3.

Compiler internals, 17 crates

STAGE	STATE	NOTE
Lexer, parser, CST, typed AST	works	Hand-written, error-recovering, trivia-preserving
Formatter	works	Pure function over the CST; lost source in 7 of the last 8 waves
HIR, name resolution, modules	works	Flat import merge; cycles legal; export filtering
InternPool: types, values, layout	works	One layout computation and one integer evaluator, shared
Sema: signatures, checking	works	E0212 to E0257; no const-eval here, by design
MIR: typed SSA	works	Block parameters, notphis; explicit bounds check and zeroing
Mid-end	5 passes	Inline, forwarding, const-prop, DCE, plus the bounds-check strip
Const-eval	works	Runs MIR through the bytecode VM
VM: register bytecode, libffi	works	No JIT; indirect calls, and malloc from its own region
Cranelift back end	works	Aggregate returns via sret; indirect calls via func_addr
LLVM back end	not started	W8 owns it
Language server	12 caps	Diagnostics, hover, goto, completion, rename, actions, hints
Neovim integration	works	Runtimepath dir, no plugin manager; 166 checks
Driver	stub	Should consume the workspace notion that now exists

Roadmap: W1 and W2 closed, W3 started

W1 Data	done	Numeric tower, wrapping ops, enum, enum_flags, union, arrays, views, cast, xx, operator overloading. Closed by ADR-0048.
W2 Flow and scope	done	for with it and it_index, labelled break, defer, using, multiple returns, named and default arguments, #scope_module. Closed by ADR-0054; ADR-0055 then closed a gap six corpus files had carried for eleven waves.
W3 Runtime core	done	context (ADR-0057), the bounds-check build setting (ADR-0058, finishing ADR-0003), indirect calls (ADR-0059), null plus a memory source (ADR-0060/0061), the allocator protocol (ADR-0062), push_context (ADR-0063), pointer arithmetic (ADR-0064), temporary storage (ADR-0065), and traps with backtraces (ADR-0066) — a trap names the frames that were live, byte-identically in both engines. Closed by ADR-0066; a source-level backtrace with inlined frames is deferred, because inlined frames have no runtime existence.
W4 Comptime	done	Delivered in sub-waves, because a 10-14 week wave cannot be verified the way a one-ADR wave can. All ten have shipped, and W4 is complete as scoped. A #run may call an imported procedure and appear in a body (ADR-0069), turning two internal compiler errors into working programs. An array length may name a constant (ADR-0070), which replaced the scheduled aggressive const folding after probing showed const-prop already did it. A type is a compile-time value (ADR-0071), which closed a silent miscompile — a type bound to a local compiled to an undefined value in a slot with no layout. Insert of a string literal lowers where it is written (ADR-0072), in the enclosing scope, every synthesized span pointing at the directive because jr-diag clamps an out-of-range offset rather than rejecting it. And a computed insert (ADR-0073) evaluates its operand at compile time and splices the text — the point sema and the VM become mutually recursive, PLAN section 5's named top risk, broken by an acyclic pre-pass that reuses the constant evaluator and re-lowers only the affected bodies rather than by salsa fixed-point recovery. An aggregate compile-time value (ADR-0074): a #run returning a struct or array interns as its element values rather than a byte image, because the pool is target-independent and an image is not. And type_info(T) (ADR-0075), reflection's first half: a type's kind, name, size and alignment, the numbers coming from the same layout_of every real layout decision uses, so reflection cannot disagree with the layout it describes. Type_Info is declared in modules/Basic in Jairs rather than inside the compiler, because it has to be spellable — no compiler-declared type can be named at all — and the resulting dependency on a declaration the compiler does not own is validated on lookup, so editing that struct is a diagnostic rather than a read of whatever now sits at the old offset. Getting there first needed a constant that may hold a string, which ADR-0074's own closing claim said was already done and was not: the fourth false scheduled dependency this project has found, and the first where the false claim was its own ADR about the very next wave. And Any (ADR-0076, ADR-0077), reflection's second half: any_of erases a value to a {Type_Info, *u8} pair and any_as reads it back, trapping unless the type matches — the erasing pointer conversion allowed only at that boundary, because a general one would make a wrong pointee type a silent wrong read. Nothing is reinterpreted, so neither back end needed a line. The checked read needed a runtime type identity the four-field Type_Info did not have — two type_info calls have different addresses, size and alignment collide, and name is unsound because a local and an imported type share a spelling — so Type_Info gained a stable id, the pool id the compiler already uses. And Type_Info's fixed-size per-kind facts (ADR-0078): a struct's field count and an array's or pointer's element type, added as flat s64 fields because a count is a number and an element type is a pool id — so they need none of the memory-ownership decision the variable-length field list does, which stays deferred. And #code (ADR-0080), unquoted source spliced into the enclosing scope — deliberately sugar over #insert, since #insert of a named constant already worked, so what it adds is no quoting and a body parsed where it is written. A Code value is declined rather than deferred: a quoted syntax tree is worth representing only once something can inspect or transform one. The same sub-wave refused a shipped silent miscompile found while probing (ADR-0079): a pointer or view inside a compile-time aggregate interned the evaluator's own address as an integer, giving 48 in one engine and a segfault in the other with no diagnostic. Out of W4's scope, each with a recorded reason: Type_Info's variable-length field list, which needs a declared static-data mechanism and is owed its own wave; a bare value coercing to Any, which needs a materialised temporary; and a #run reading another file's constant, which now reports itself rather than an internal error.
W4.5 Pattern matching	done	switch with exhaustiveness checking, a bare dot-member as a case (settling ADR-0041 §2 step 5), and a tagged variant type (ADR-0067, ADR-0068). Reordered ahead of W4 after checking showed its stated dependency on comptime was a want rather than a need. The variant follows ADR-0045 §1's own instruction — a different declaration form, not a change to union — and union is untouched, still untagged and still one word smaller.
W5 Polymorphism	done	COMPLETE in fifteen sub-waves (ADR-0081 to ADR-0097). The last piece, #bake_arguments specialisation, lowers a declaration to a REAL procedure — a clone with the baked parameters dropped, their literals substituted and the kept ones remapped, which is the same machinery \$N instantiation uses, so W5 ends on a reuse rather than a new mechanism. A baked value must be a literal: ADR-0096 planned to use the const-eval pre-pass and building it showed that pre-pass runs AFTER lowering. #bake_arguments has its surface (ADR-0096) — a partial application producing a specialised procedure; its operand is a call so the named-argument spelling is the ordinary one, and its specialisation is refused E0276 pending the last W5 sub-wave. That refusal replaced a leaked gap report (the compiler could not lower main, please report it) which is right for an unknown gap and wrong for a named one. #modify is COMPLETE (ADR-0093/0094/0095): a compile-time predicate over an instantiation runs when a call binds the type variables, and a false refuses that instantiation (E0275) — so a template enforces its own constraints in code. It is hosted in file_mir, the only place with the expanded tree, its MIR and the VM, and rejections ride the existing diagnostics channel so it needed no new query. A predicate that fails to RUN is deliberately not a rejection. E0274 was retired when the predicate began running — the fourth by-design refusal raised then lifted. #modify has its surface (ADR-0093): a compile-time predicate over an instantiation, guarding a template in code rather than a comment. The block parses (the one procedure attribute

carrying a block) and formats with its body; a call is refused E0274 pending evaluation, because a parsed-and-ignored predicate would accept calls the author rejected — ADR-0058 rule for the third time. Its evaluation is designed and deferred. `type_info(T)` now reflects a BOUND type variable (ADR-0092) — a `$T` procedure can ask its own bound type size, field count or identity, each instantiation seeing its own. That was found missing while designing `#modify`, whose predicate needs exactly it, and fixing it also turned a sixth leaked internal error into working code. The `#expand SPLICE` works (ADR-0091): a call splices the macro body into the caller scope, so a macro can modify the caller local — deliberately unhygienic like `Jai`. A generated prelude binds each argument once (substituting per use would re-evaluate a side-effecting argument), and expression position gets a generated result local so one mechanism serves both. The MIR shows no calls at all. Refused by design: an early return (E0273), a void macro in expression position, a cross-file call (E0272, which had been reaching the VM as an internal error). `looks_like_proc_signature` needed `#expand` too — the token-set trap for the fifth time, since a void macro reaches neither arrow nor brace. `#expand` macros have their surface (ADR-0090): a macro parses, formats and checks like any procedure, and a call is refused E0272 pending the splice — a refusal that ships WITH the surface, because without it `#expand` was accepted and silently ignored (a macro behaved as an ordinary procedure). `jr-fmt` dropped `#expand` on the first run, caught by gate 5. `$N` comptime-value parameters are complete — surface, instantiation, and `[N]T` sized by one (ADR-0089), where two instantiations get genuinely different array types from one declaration. They work end to end (ADR-0087 surface, ADR-0088 build): `make :: ($N: s64)` called as `make(5)` evaluates the argument via the same acyclic pre-pass `#insert` uses, and appends a concrete procedure with `N` baked into the body — parameter list drops the `$N`, each reference to `N` becomes a literal. Two calls at the same value dedupe (ADR-0005 extended to values), distinct values instantiate separately. Mixed comptime+runtime params (scaled `:: ($N: s64, factor: s64)`) pass only the runtime one at the call site — a per-call arg-mask filters at MIR, teeth-checked (disabling it makes the verifier catch an arity mismatch). E0271 refuses a non-constant argument at the call's span. Before that, `$T` procedures work end to end (ADR-0081-0084): a `$T` parameter is inferred from the call — directly or through a pointer or view — instantiated once per distinct tuple of bound types, checked per instantiation, and run as an ordinary procedure in both engines, so nothing polymorphic survives to the back end. And polymorphic structs (ADR-0085, built per ADR-0086): `Box :: struct($T) { value: T; }` used as `Box(s64)` is a type constructor, and `Box(s64)` and `Box(bool)` are distinct types from one declaration with substituted fields and layouts, told apart in the pool by the type argument in the key the way `[2]s64` and `[3]s64` are. It changed the pool's most load-bearing invariant — a struct's identity was its declaration site — and was landed in two commits, a zero-behaviour-change representation refactor proven by an unchanged snapshot and test count, then the parameterised behaviour, so a half-built type-identity change could not hide a miscompile. W5 is done; next is W6 Metaprogram then W7 Stdlib (modify, `bake_arguments`, `expand`), and the deferred struct pieces (inference through `Box($T)`, using on one, cross-file, recursive `List($T)`), each a refusal today rather than a gap.

W6 Metaprogram

in progress OPEN, four sub-waves shipped, and the wave headline claim is MET: a metaprogram can find declarations by note and generate code for each one. `@note` attaches metadata to a declaration (ADR-0098) — `@deprecated`, `@requires "x"` — its own node kind rather than a generic attribute, because a note is DATA for a metaprogram while the directives are INSTRUCTIONS to the compiler. `has_note` and `note_value` READ it at compile time (ADR-0099), folded in sema with no VM and no new query — unlike `type_info`, which folds later because it needs a layout; a note answer is in the HIR the checker already holds. The first argument is the declaration itself rather than its name as text, so a misspelling is an unresolved name instead of a silent false, which is the same silence the formatter dropped notes had. An absent note answers false and empty and is NOT an error: asking whether a note is present is the point, the opposite call from any `_as` which traps, and the difference is that any `_as` would otherwise return garbage while this returns the truth. `noted_count` and `noted_name` QUERY the file without naming declarations (ADR-0100), in declaration order — the one order a reader can predict, since a name sort rennumbers every unrolled index when a declaration is inserted and a hash order makes one program answer differently between runs. And `noted_insert GENERATES` (ADR-0101): one template emitted once per noted declaration, hash standing for each name, spliced by `#insert` — so a single line generates a call to every noted procedure. The loop lives inside the FOLD, and that corrects ADR-0100 scope the way ADR-0094 corrected ADR-0093: folding cannot take a for variable, but that forbids a loop in the PROGRAM and says nothing about a loop inside the FOLD — and for generation the fold is the only thing that can work at all, since generated code must exist before checking, so a run-time loop could not declare a procedure or emit a statement. Probing found that `#insert note_value(f, "gen")` ALREADY worked, a note payload spliced as code, shipped and undocumented. Two refusals were leaked errors until this wave probed them: `==` on two strings reached the VM as expected a scalar found an aggregate (E0278), now refused for every aggregate the way a view equality already was, by a structural test since size and alignment cannot tell an `s64` from a two-field struct of `s32s`. And a latent miscompile was fixed (ADR-0101): a folded value keyed by `ExprId` is STALE once a body expands, because a computed `#insert` rennumbers every id after its splice — so with two computed inserts the second value landed on a different expression, putting a string on an arithmetic operand, and it surfaced as a VERIFIER PANIC rather than a diagnostic. That is the sharpest well-typed placeholder this project has had: the two earlier ones were placeholders that happened to be legal values, while this is a genuine value from the same program merely attached to the wrong expression, so nothing in the type system can see it is wrong. Remaining in W6: run-time INSPECTION — a loop reading declarations as values — which needs a compiler-emitted static-data table and lifts `Type_Info` variable-length field list at the same time, plus `#run build()` build scripts, plugin hooks and workspaces.

W7 Stdlib

in progress OPEN, two modules shipped, and both were named by a refusal rather than by a checklist. `String` (ADR-0103) is equal, compare, starts_with, ends_with, find, contains, byte_at, is_empty — NONE of which allocate. It exists because ADR-0099 refused `==` on two strings: a string is `{data, count}`, so same storage and same contents are both plausible readings, and picking one silently would make the other a bug that looks like working code. That refusal own reasoning named the fix, comparing contents needs a byte loop, which is a library job rather than an operator — an `==` that looped would also be the only implicitly-looping operator in the language. So `equal(a, b)` is what E0278 help was pointing at. It is its OWN module rather than more of Basic, and the deciding argument is not size: Basic is imported by every program so anything in it taxes all of them, but more importantly adding to Basic would mean nothing ever tested that TWO modules can be imported at once, which every module test to date failed to do. Nothing allocates deliberately: `concat` and friends need somewhere to put a result, and while the mechanism exists (context allocator, temporary storage) the CHOICE between always-the-context-allocator, an explicit parameter, and always-temporary does not — and settling it in passing is how a library acquires an accidental convention. `byte_at` exists because `s.data[i]` does not compile, a `*u8` not being indexable, so reading a byte takes pointer arithmetic and a cast; out of range answers -1 rather than trapping, unlike an array index, because an array bound is known to the compiler so passing it is a mistake while scanning until the bytes run out is an ordinary loop. `Sort` (ADR-0104) is the second and the first POLYMORPHIC library code, so the first that depends on W5 rather than coexisting with it. Three language facts were probed before a line was written and all hold: a view parameter is mutable through the callee, a `$T` infers through a view, and a procedure pointer can be passed and called. The CALLER supplies the comparison, and that is

a language fact rather than a taste: resolving an operator inside a \$T template against the instantiated type is a lookup instantiation does not do — operator < exists and #modify can REJECT an instantiation, but nothing can SELECT an implementation per instantiated type, which would be operator-bounded polymorphism and belongs to whichever wave decides how a template states its requirements. Insertion sort, O(n squared) said plainly: STABLE which quicksort is not, needs NO storage where a merge sort would allocate and allocation is exactly what String declined to decide, and short enough to read — which matters for the first sorting routine in a language whose suite compares two independent engines. A faster algorithm is W8 with a benchmark behind it. Writing Sort found TWO leaked internal errors, which is the argument for a standard library written in the language paying out twice in one sub-wave. An imported procedure used as a VALUE reported this compiler has a gap please report it, for a legal program — the value was representable all along since ImportedProcs had resolved it to a ProcRef and a DeclId carries a file id, so what was missing was a three-line bridge; the local arm own comment even said a cross-file one is refused by that arm, so the refusal was KNOWN and undocumented, recorded as intended in a comment while users saw a bug report. And a call to an imported TEMPLATE leaked no routine for file 2 proc 0: cross-file instantiation is deferred, but the refusal did not exist because a \$T parameter type is ERROR and ERROR matches anything, so the call type-checked. It is now E0268, carried across the boundary by template_names shaped exactly like macro_names, and the diagnostic NAMES THE WORKAROUND with a corpus file checking that the workaround works — a refusal is only as good as its escape route. It survived because nothing in the corpus had ever imported a polymorphic procedure. Both bugs were hiding behind a stale comment that said something checkable nobody had checked. So Sort sort_ints and ints_sorted are NOT conveniences today: they are the only way an importer can use a polymorphic module. Remaining in W7: the allocating half of String once the allocator convention is decided, a merge sort and binary search, a dynamic array and hash table which both want W5 polymorphic structs, Math, Random, File, Process, Thread, Time, Socket, JSON, Compiler.

W8 Performance	not started	LLVM via inkwell for release builds, inliner maturity, struct-of-arrays, SIMD, parallel sema. Also owns the compile-throughput number, still unpublished.
W9 Tooling depth	not started	Semantic tokens are the one LSP capability still missing; the other twelve landed early.
W10 Graphics, in Jairs	not started	Window creation through foreign calls, Metal then Vulkan, immediate-mode 2D.

What this session did, and how each wave was scoped

FORTY-NINE WAVES SHIPPED

ADR-0049 through 0097: for and defer, using, aggregate returns, multiple returns, named and default arguments, scope visibility, imported constants, float constants, context, the bounds-check build setting, indirect calls, null plus a memory source, the allocator protocol, push_context, pointer arithmetic, temporary storage, trap backtraces, switch, tagged variants, compile-time run across files and in a body, an array length from a constant, a type as a compile-time value, insert of a literal and a computed string, an aggregate compile-time value, type_info and Any, code splicing, \$T procedures, polymorphic structs, and \$N comptime-value parameters with array lengths.

Test count 900 to 984. Corpus 116 to 194 files. Neovim checks 103 to 166.

W2, W3, W4.5, W4 and W5 are all closed, and W6 is open. W5 shipped fifteen sub-waves: \$T procedures, polymorphic structs, \$N comptime-value parameters **and** their instantiation. A \$N call make(5) evaluates the argument via the same acyclic pre-pass #insert uses (ADR-0073), and appends a concrete procedure with the value baked into the body — the parameter list drops the \$Ns, each reference to N becomes a literal. Two calls at the same value dedupe, distinct values instantiate separately, and mixed comptime+runtime params pass only the runtime ones at the call site. A per-call arg-mask filters at MIR, teeth-checked (disabling it makes the verifier catch an arity mismatch). **W6 Metaprogram** and **W7 Stdlib** are both open. W6 headline claim is met — notes are attached, read, queried and generated from — and a build script names its own artefact. W7 has String and Sort.

Each wave was scoped by writing the feature and seeing what the compiler refused, not by reading the handoff. That found gaps the handoff had missed in both directions: a proc-pointer struct field it called absent (it worked), and a void-returning proc-pointer type it never mentioned (unspellable, and blocking).

THE THING WORTH YOUR DECISION

Every wave is now committed as it greens, on its own feat/ branch, which closed the risk this box used to name. Nothing has been merged: main is still at ec150a5, and twenty-three waves sit on stacked branches ahead of it. That is a decision rather than a gap — merging needs your say-so, and the per-wave commits mean no work is at risk while it waits.

The one open slice criterion is still a verified Linux x86-64 CI run, which needs a push.

TWO CLAIMS THE CODE DISPROVED

An ADR corrected mid-wave. ADR-0060 §4 asserted the VM dereferences a malloc'd pointer via libffi. Running the corpus file the same ADR promised would pass disproved it: the VM's memory is a linear region and a host address is not an offset into it. ADR-0061 corrects it — the VM allocates from its own region, native calls libc, the bits differ and nothing observes them.

A diagnostic that could not be acted on. Installing an imported #foreign procedure into an allocator field reported “expected (s64) -> *u8, found (s64) -> *u8” — the same text twice, because the types differ only in an invisible calling convention. It is E0256 now, the code that says to wrap it.

Sources: PLAN.md §1.5 and §7, the ADR directory, docs/decisions/DECISIONS.md, and the six gates run today. Every number was measured rather than carried forward — the test count from a full workspace run, the corpus count from a file walk, the diagnostic codes from the const E0nnn definitions across the crates, and the editor-check count from a real verify.lua run. Rebuild with typst compile jairs-dashboard.typ.